

考虑一个语言 L 包含 n 个词, $L = \{w_1, \dots, w_n\}$ 。一个 2-gram 语言模型可以看成是以词为下标的矩阵, 每个元素 P_{ij} 表示在语言中连续出现 w_i 和 w_j 的概率:

$$P_{ij} = \mathbb{P}(w_i \rightarrow w_j)$$

注意, 这是一个概率矩阵而不是转移矩阵, 满足 $\sum_{ij} P_{ij} = 1$, 而不是 $\sum_j P_{ij} = 1$.

现在我们考虑在这样的语言模型下计算连续当量。记 w_i 的编码为 c_i , 并且这个编码的第一码和最后一码分别为 s_i 和 t_i , 满足 $s_i, t_i \in \Sigma$ 是某个字母表。记当量的函数 $f: \Sigma \times \Sigma \rightarrow \mathbb{R}$ 是将两个字母映射为一个实数的函数。

连续当量的定义为:

$$E_{\text{cont}} = \sum_{ij} P_{ij} f(t_i, s_j)$$

也就是说, 对每个可能的词间连接来统计加权平均的当量。这个定义的问题在于, P 是一个非常大的矩阵, 即使使用稀疏矩阵的方式加以压缩, 也至少有数十万甚至数百万元素。考虑到一般来说词的数量只有几万, 在方案评测和计算中考虑连续当量的性价比是不高的。下面我们来研究如何降低这里的计算量。

注意到, 如果认为 $P_{ij} \approx p_i p_j$, 也就是忽略任何语言模型中的相关性, 认为转移概率正比于词的频率乘积, 那么上式可以极大化简:

$$\begin{aligned} E_{\text{cont}} &\approx \sum_{ij} p_i p_j f(t_i, s_j) \\ &= \sum_{ab} \sum_{t_i=a} \sum_{s_j=b} p_i p_j f(a, b) \\ &= \sum_{ab} \mathbb{E}(t = a) \mathbb{E}(s = b) f(a, b) \end{aligned}$$

上式中我们不是对每个 (i, j) 分别求和, 而是考虑对所有的键位组合 (a, b) 求和, 那么就只需要计算「末码为 a 的概率」和「首码为 b 的概率」, 然后让当量在这个概率下加权平均即可。这两个概率都比较好计算, 最后的求和也仅相当于一个 $|\Sigma|$ 大小的矩阵向量乘积。不过, 这种方法没有考虑语言模型, 所以不是我们想要的。不过实际上, 这个就是各种双拼方案的评测网站上所采用的「连续当量」的定义——可能叫「韵声当量」更合适。

但是，注意到任何矩阵都可以基于奇异值分解来进行低秩近似：

$$P \approx \sum_{\alpha} \sigma_{\alpha} u^{(\alpha)} v^{(\alpha)T}$$

上式中我们把这个矩阵描述为几个（实践中不需要很多，< 10）秩 1 的矩阵之和，因此就有

$$P_{ij} \approx \sum_{\alpha} \sigma_{\alpha} u_i^{(\alpha)} v_j^{(\alpha)}$$

用这个近似我们可以得到：

$$\begin{aligned} E_{\text{cont}} &\approx \sum_{ij} \sum_{\alpha} \sigma_{\alpha} u_i^{(\alpha)} v_j^{(\alpha)} f(t_i, s_j) \\ &= \sum_{\alpha} \sigma_{\alpha} \sum_{ab} \sum_{t_i=a} \sum_{s_j=b} u_i^{(\alpha)} v_j^{(\alpha)} f(a, b) \\ &= \sum_{\alpha} \sigma_{\alpha} \sum_{ab} \mathbb{E}_{u, \alpha}(t = a) \mathbb{E}_{v, \alpha}(s = b) f(a, b) \end{aligned}$$

这里我们把内层的求和看成是关于 u 或者 v 向量的「加权平均」（这里不严格，因为它们是 2-范数归一化的，但不是 1-范数归一化的），就可以像前面的近似一样通过仅计算加权平均和小矩阵的乘积来获得一个比较好的近似值。 α 的数量是可变的，用户可以指定取奇异值的前几个，从而渐进地获得越来越好的近似。